FINOPS-02: Forecasting and Anomaly Detection on DPS Consumption

> **Series:** FINOPS — Cost Management & FinOps | **Reference:** 02 — Forecasting and Anomaly Detection on DPS Consumption | **Created:** May 2026 | **Last Updated:** 10/14/2026

Overview

Knowing *current* consumption (covered in FINOPS-01) is operational. Knowing *projected* consumption — and being alerted when the trajectory breaks — is what turns FinOps from a monthly report into a daily control loop. This entry walks through both the native Dynatrace surfaces (Account Management Cost Monitors and Budget Alerts) and the in-tenant DIY surfaces (Davis Predictive AI on `dt.billing.*` series, plus Workflow-driven burn-rate alerts).

Three layers of cost visibility: *Operational* (what's happening right now), *Tactical* (where are we trending this week / this month), *Strategic* (will we hit our annual commit?). Each layer pairs with different surfaces, different cadences, and different audiences. In community practice, treating them as one thing — "the cost dashboard" — is a frequent reason cost programs never escape monthly snapshots.

Native first, DIY where native is silent. Cost Monitors and Budget Alerts cover most strategic and tactical use cases out of the box. DIY DQL forecasting is for the operational layer where you want a custom signal (per-bucket trajectory, per-team burn, per-capability projection), or when the native surface doesn't yet exist for the cut you need.

> **Scope:** Dynatrace SaaS on DPS. Davis Predictive AI's `timeseries-forecast` and `seasonal-baseline-anomaly-detector` analyzers run on any time-aligned metric series, including `dt.billing.*`. The portal-side Cost Monitors evolve sprint-to-sprint — verify specific UI affordances against current docs at procurement-review time.

Table of Contents

1. Short Answer
2. Three Layers of Cost Visibility
3. Native — Account Management Cost Monitors
4. Native — Budget Alerts
5. DIY — DQL Trend Queries with `makeTimeseries`
6. DIY — Davis Predictive AI on `dt.billing.*`
7. DIY — Workflow Burn-Rate Alerts
8. Worked Example — End-of-Month Projection
9. Worked Example — Capability-Level Anomaly Alert
10. When to Use Native vs DIY
11. Recommended Approach
12. Summary and Next Steps

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS on DPS. Davis Predictive AI is available platform-wide. Cost Monitors and Budget Alerts require Account Management portal access (separate IAM scope from tenant IAM).
Permissions	`storage:metrics:read` for `timeseries dt.billing.*` queries; `davis:analyzers:execute` for `timeseries-forecast` and `seasonal-baseline-anomaly-detector`; `automation:workflows:write` for workflow-based burn-rate alerts; account-level admin or finance role for Cost Monitors and Budget Alerts configuration.
FINOPS-01 understanding	This entry assumes familiarity with `dt.billing.*` vs `dt.system.events` and the per-capability schema.
Audience	Platform / Observability Lead (DIY surfaces), Executive / Procurement (native surfaces and §11 recommendation).

1. Short Answer

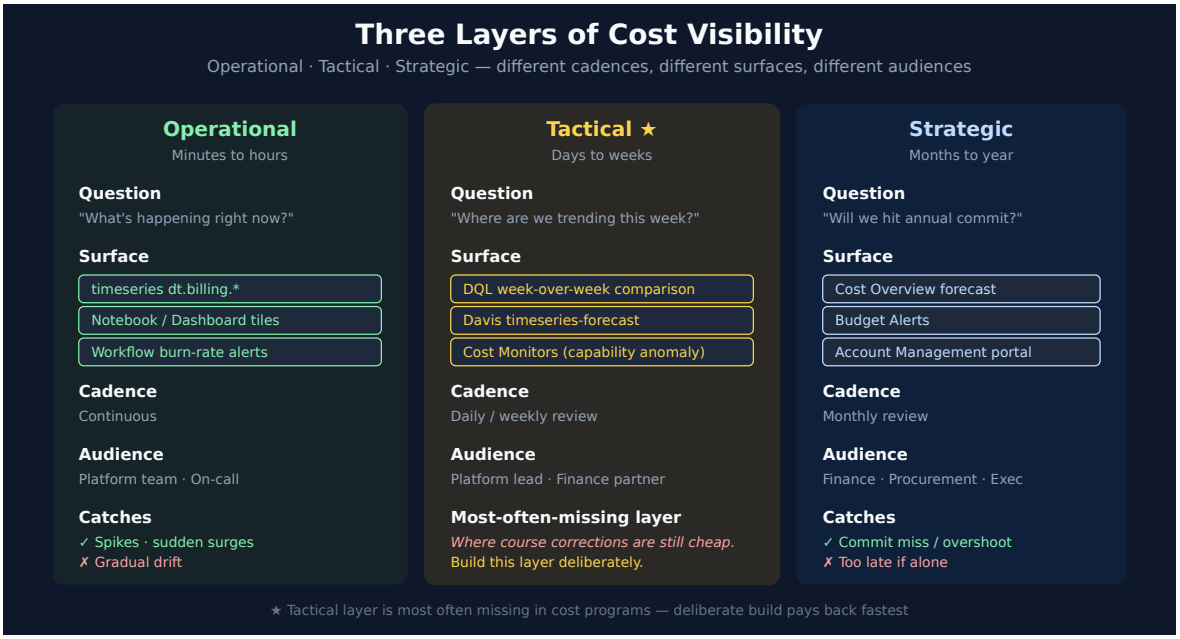
Question	One-line answer
Will we hit our annual commit?	Account Management → **Cost Overview** → forecast line on Cost Overview. Native, automatic, billing-period-aligned.
Alert me if we're on course to overshoot	Account Management → **Budgets**. 75% / 90% / 100% of the annual commitment are pre-configured; add account-, environment-, or capability-level budgets. Notifications via the Notification Center and email, evaluated daily.
Alert me if a *specific capability* is spiking	Account Management → **Cost Monitors** — on automatically; they check each capability in each environment daily.
Forecast per-bucket / per-team consumption	DIY — `timeseries dt.billing.*` + Davis `timeseries-forecast` analyzer.
Alert when per-bucket burn exceeds threshold	DIY — Workflow with scheduled DQL trigger + Davis anomaly analyzer or static threshold.
7-day / 30-day trend visualization	DIY — `timeseries dt.billing.<capability>.usage` for host-based; `fetch dt.system.events | makeTimeseries` for byte/count-based.
Day-over-day or week-over-week comparison	DIY — DQL with two parallel time ranges; pattern in `dt-dql-essentials` skill.

The headline: native surfaces (Cost Monitors, Budgets) cover commitment-level visibility automatically. Dynatrace's own guidance draws the line where the built-in forecast stops — it *"only goes as far as environment and capability level"* — and points to DQL for growth rates, 30/60/90-day scenarios, cost-center or team breakdowns, and days-until-a-budget-fires. That is the DIY layer: DQL + Davis analyzers + Workflows.

> **Sources:**
>
> - Forecast costs with run-rate projections (DT docs) — *"Account Management already gives you a built-in forecast, but it only goes as far as environment and capability level."*
> - Budget alerts (DT docs) — *"Dynatrace notifies you as actual or forecasted consumption approaches each threshold (defaults: 75%, 90%, 100%)."*

> - Cost monitors (DT docs) — *"Cost monitors inspect costs for each capability in each environment daily and notify you when a capability exhibits an unexpected increase"*
> - DQL reference (DT docs)

2. Three Layers of Cost Visibility



In community practice, a working FinOps program operates at three time horizons, and conflating them is a common failure mode — "the cost dashboard" usually means one of the three was built and the other two are missing. The three layers are this entry's framing, not a Dynatrace taxonomy.

Layer	Time horizon	Question	Best surface	Cadence
Operational	minutes to hours	What's happening right now?	`timeseries dt.billing.*` in a Dynatrace dashboard; Workflow burn-rate alerts	continuous
Tactical	days to weeks	Where are we trending this week / this month?	DQL forecast (Davis `timeseries-forecast`); Cost Monitors anomaly alerts	daily / weekly review

Layer	Time horizon	Question	Best surface	Cadence
Strategic	months to year	Will we hit our annual commit?	Account Management → Cost Overview forecast; Budget Alerts	monthly review

Why all three matter

- **Operational alone** catches spikes but misses gradual drift. A team that doubles ingestion volume over six weeks looks fine hour-by-hour but blows the annual commit.
- **Strategic alone** catches the commit miss but only after most of the year has been spent. The conversation with the team that doubled ingest happens in November, when there's no runway left.
- **Tactical alone** is where most cost programs actually live, and it's the layer most likely to be silent — daily / weekly trend is exactly the cadence at which course corrections are still cheap.

In community practice, the layer most often missing is tactical. Operational dashboards get built because they're easy. Strategic Cost Overview is automatic. The weekly burn-rate alert that says "this team's ingest is 40% above its 4-week baseline" is the layer that takes deliberate work — and the layer that prevents the November conversation. Verify against your own program before treating it as universal.

3. Native — Account Management Cost Monitors

Cost Monitors are Dynatrace's native anomaly-detection surface for DPS consumption. They live in the Account Management portal (not in the tenant) and work on the same cost data that powers the Subscription overview.

What they do

- Are **on by default** — enabled automatically for every DPS account, notifying license administrators with no configuration
- Watch the **forecast**: notify when the end-of-period forecast crosses a threshold (default 95% of the commitment) or jumps week-over-week
- Watch **daily cost per capability per environment**, and raise a cost event when a capability shows an unexpected increase

- Use **linear forecasting** over the past month of consumption, weighting recent days more heavily — no forecast is shown until 15 days of data exist
- Deliver through the **Notification Center**, **email** (up to 50 recipients), and the **API**

What to configure

1. **Recipients.** License administrators are notified by default. Add the people who can act — a Cost Monitor that alerts a finance distribution list is information; one that reaches the platform team owning the OpenPipeline pipeline behind the spike is action. The Notification Center can route different alert categories to different teams.
2. **Forecast thresholds.** Set the week-over-week threshold and the total-forecast threshold (default 95%) to match how early you want warning.
3. **Expect ramp-up noise.** Dynatrace's own guidance is that week-over-week forecast notifications can be ignored during the deployment phase, when rising usage is expected. In community practice, review the cost events weekly for the first month and note which ones were actionable before changing thresholds.

When Cost Monitors are silent

Cost Monitors operate at the capability-and-environment level — they catch "Logs ingest is way up in production" but not "the `payments-team` bucket is way up while the `marketing-team` bucket is steady." For per-bucket / per-team anomaly detection, see §6 (Davis Predictive AI in DQL).

> **Sources:**
>
> - Cost monitors (DT docs) — *"Cost monitors are enabled automatically for all DPS accounts and will notify license administrators by default without configuration"*; *"Cost monitor algorithms use linear forecasting techniques to predict future usage from the past month of consumption data."*; *"If you're still in the deployment phase of Dynatrace, you can ignore this kind of notification, as increased weekly usage is expected."*
> - Control (DT docs) — *"Cost monitors detect unexpected changes as they happen and route alerts to the team that owns the consumption."*

4. Native — Budget Alerts

Where Cost Monitors are anomaly-detection ("is this period unusual"), **Budget Alerts** are threshold-based ("are we on track to hit the commit"). Both surfaces live in Account Management, often configured by the same person, but they answer different questions.

> Neither Cost Monitors nor Budget Alerts have a Settings 2.0 schema or Terraform resource — they're Account Management API objects, confirmed not to exist alongside the rest of the repo's schema catalog in **AUTOM-02**. Don't go looking for a `builtin:` schema for cost config; it isn't there.

What they do

- Compare actual **and forecast** consumption against a threshold you set at the account, environment, or capability level — a fixed amount or a percentage of the annual commitment (up to 20 active budgets)
- Ship with defaults — every DPS account gets 75% / 90% / 100% of the annual commitment, notifying license administrators; adjust them or add tiers for earlier signal
- Notify through the Notification Center and email; each budget has its own recipient list
- Evaluate once a day, after the daily cost calculation (usually by 12:00 UTC), and email only on the day a threshold is first exceeded

What to configure

1. **Scope budgets where one area drives cost.** Keep the account-level defaults and add environment- or capability-scoped budgets — Dynatrace's own examples are 10% of total subscription costs for a sandbox environment, and a fixed USD 1,000 threshold for a new capability.
2. **Pick threshold tiers based on time-to-react.** The default is 75/90/100 of the annual commitment, enabled automatically. In community practice, many programs add an earlier custom tier (e.g., 50%) for more lead time; tight-commit customers sometimes run 25/50/75.
3. **Tier the notification escalation.** In community practice: 50% → platform team; 75% → platform team + finance; 90% → procurement + leadership. The threshold tiers map to who needs to act.

4. **Remember the daily cadence.** Budgets have no intra-day alerting — a same-day spike surfaces the next evaluation at the earliest. Pair them with Cost Monitors and the DIY alerts below when you need faster signal.

When Budget Alerts are insufficient

Budget Alerts are coarse — account-, environment-, or capability-level, evaluated daily. They don't decompose to per-team / per-bucket attribution at all. Pair them with Cost Monitors (capability-level anomalies) and DIY DQL queries (team-level attribution).

> **Sources:** Budget alerts (DT docs) — *"A Dynatrace Platform Subscription automatically enables three initial budget thresholds that notify license administrators when total account level consumption reaches 75%, 90%, and 100% of the annual commitment."*; *"You can get notifications in the Notification Center and via email."* Forecast costs with run-rate projections (DT docs) — *"There is no intra-day alerting."*

5. DIY — DQL Trend Queries with `makeTimeseries`

For host-based capabilities, prefer `timeseries dt.billing.*` (covered in FINOPS-01 §5). For byte- and count-based capabilities, use `fetch dt.system.events | makeTimeseries` to build a time-aligned trend series:

```
// Daily log ingest GiB per bucket — 30-day trend
fetch dt.system.events, from:-30d
| filter event.kind == "BILLING_USAGE_EVENT"
| filter event.type == "Log Management &amp;amp; Analytics - Ingest &amp;amp; Process"
| dedup event.id
| fieldsAdd gib = toDouble(billed_bytes) / 1073741824
| makeTimeseries daily_gib = sum(gib), by:{ usage.bucket }, interval:24h
```

Worked example — week-over-week comparison via two parallel time ranges:

```
// Week-over-week log-ingest comparison — this week vs last week
fetch dt.system.events, from:-14d, to:-7d
| filter event.kind == "BILLING_USAGE_EVENT"
| filter event.type == "Log Management &amp;amp; Analytics - Ingest &amp;amp; Process"
| dedup event.id
| summarize { last_week_gib = sum(toDouble(billed_bytes)) / 1073741824 }, by:{
usage.bucket }
| lookup [
    fetch dt.system.events, from:-7d
    | filter event.kind == "BILLING_USAGE_EVENT"
    | filter event.type == "Log Management &amp;amp; Analytics - Ingest &amp;amp;
Process"
    | dedup event.id
    | summarize { this_week_gib = sum(toDouble(billed_bytes)) / 1073741824 },
by:{ usage.bucket }
  ], sourceField:usage.bucket, lookupField:usage.bucket
| fieldsAdd this_week_gib = lookup.this_week_gib
| fieldsAdd delta_pct = round((this_week_gib - last_week_gib) / last_week_gib
* 100, decimals: 1)
| fields usage.bucket, last_week_gib, this_week_gib, delta_pct
| sort delta_pct desc
```

Buckets with high positive `delta_pct` are growing fast and merit a closer look. The pattern generalizes to month-over-month (`from:-60d, to:-30d` and `from:-30d`), day-over-day (1d), or any other comparison window.

> **Sources:** DQL `makeTimeseries` (DT docs). The week-over-week `lookup` pattern is canonical in `dynatrace-dql-examples` (parameter-level optimization recipes). Both queries are syntactically valid; execution-dependent on having ≥14 days of tenant history. The daily trend query re-executed 10/05/2026 with `interval:24h` (no notifications; `1d` raised a deprecated-calendar-duration notice).

6. DIY — Davis Predictive AI on `dt.billing.*`

Davis Predictive AI's forecast analysis runs against any numeric time series, which includes the `dt.billing.*` family. This is the in-tenant equivalent of the Account Management Cost Overview forecast, but at a finer cut — per bucket, per cost center, per capability.

How to invoke it

The analyzer takes a time-bucketed input series (at least 14 data points) and returns point predictions plus a prediction interval. Two access paths:

1. **Notebook** — run a forecast analysis on a `timeseries` query result.
2. **Workflow task** — a Davis analyzer action with the analyzer `dt.statistics.GenericForecastAnalyzer` and the DQL query as input. (Earlier revisions named `dt.statistics.forecast`, which is not an analyzer; the name here was read from the tenant's analyzer list on 09/28/2026.)

Pattern — forecast per-capability host-hours for the next 30 days

Input series (28-day history, hourly):

```
// Forecast input — 28-day Full-Stack Monitoring history at hourly granularity
timeseries hourlyUsage = sum(dt.billing.full_stack_monitoring.usage, rate:1h),
  from:-28d, interval:1h
```

Feed this query into the forecast analyzer. The horizon is counted in data points and capped at 600, so at hourly granularity the longest horizon is 25 days — for a full 30 days, rebuild the input at `interval:24h` and forecast 30 points. Output: forecasted usage per step with a prediction interval (default coverage probability 0.9).

What to do with the output

- Sum the forecast over the remaining month → end-of-month projection
- Compare projection × rate-card conversion against the monthly budget
- Surface in a Workflow that posts to Slack / email if the projection exceeds threshold

Pattern — anomaly detection on consumption (seasonal baseline)

For "is *this* hour unusual given the seasonal pattern," use the seasonal baseline anomaly detection analyzer (`dt.statistics.anomaly_detection.SeasonalBaselineAnomalyDetectionAnalyzer`) instead of the forecast analyzer. Same input shape; different output — a confidence band learned from the series' seasonality, whose width is set by the `tolerance` parameter (0.1–10, default 4), and the points that violate it.

When Davis Predictive AI on consumption is wrong

Forecasting assumes stationary or seasonally-stationary input. Consumption time series are *not* stationary during onboarding phases — a new tenant ramps up over weeks, adoption changes the baseline, new buckets get created. Treat the first 2–4 weeks of any new tenant or major rollout as too-noisy-to-forecast: the analyzer needs 14+ days of input to detect weekly seasonality, and Dynatrace's own built-in forecast waits for 15 days of data and is to be treated with caution below 30.

> **Sources:**
>
> - Predictive AI analysis (DT docs) — *"The forecast analysis predicts future values of any time series of numeric values."*; minimum 14 data points, horizon 1–600, default coverage probability 0.9, weekly seasonality needs 14+ days of input
> - Seasonal baseline (DT docs) — tolerance range 0.1–10, default 4
> - Use the built-in forecast (DT docs) — the 15-day minimum and the under-30-days caution for the built-in forecast
> - Workflows (DT docs). Analyzer names read from the validation tenant's analyzer list (`dtctl get analyzers`), 09/28/2026.
> - **Derived:** applying the analyzer to `dt.billing.*` follows from its any-numeric-series scope; the 2–4 week settling window combines the 14-day seasonality requirement with the built-in forecast's 15/30-day guidance.

7. DIY — Workflow Burn-Rate Alerts

Workflow burn-rate alerts combine the DQL trend queries (§5) and Davis forecast / anomaly (§6) into an active control loop: scheduled execution, threshold evaluation, notification.

In community practice, two recipes cover most needs. Their DQL building blocks — week-over-week growth and days-until-a-budget-fires — are the ones Dynatrace documents in its run-rate tutorial.

Recipe — daily per-bucket burn-rate workflow

1. **Trigger:** Schedule — daily at a consistent time (e.g., 09:00 in the platform team's time zone).
2. **Task 1 — DQL query:** week-over-week per-bucket comparison (the §5 pattern).
3. **Task 2 — Filter / threshold:** select buckets with `delta_pct > 50%` (a configurable threshold).
4. **Task 3 — Notify:** post to Slack / email with the list, linking back to the FINOPS-01 §10 attribution query for drill-down.

Recipe — capability-level Davis-forecast workflow

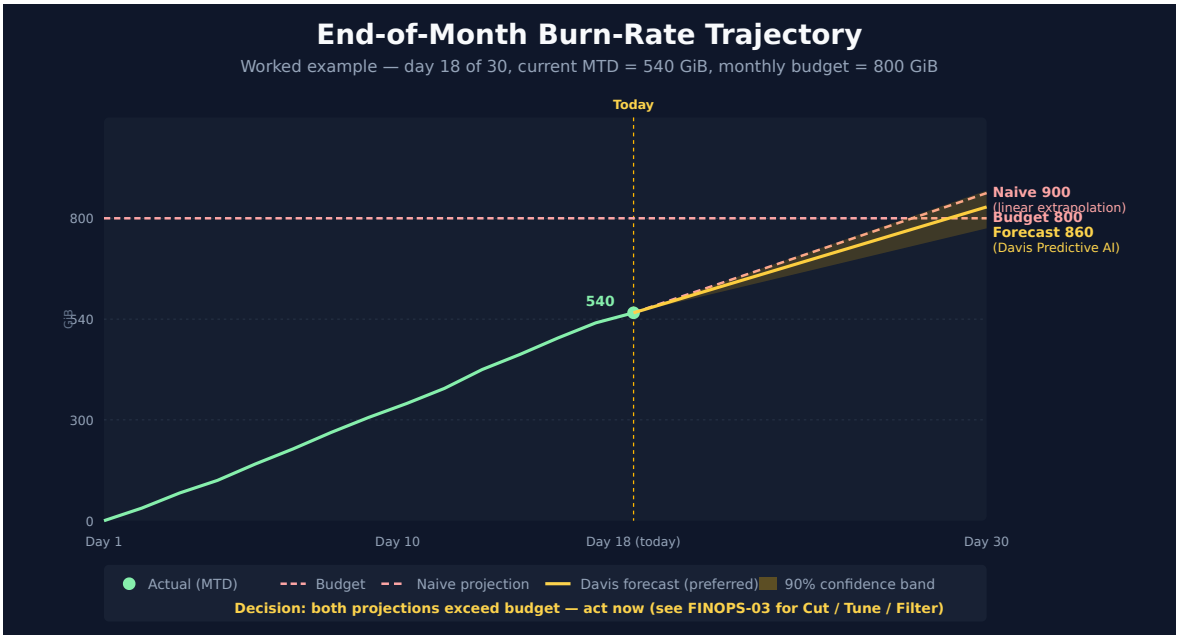
1. **Trigger:** Schedule — weekly (Monday 09:00 covers the prior-week trajectory).
2. **Task 1 — DQL query:** 28-day hourly history of each `dt.billing.<capability>.usage`.
3. **Task 2 — Davis-forecast analyzer:** project forward (the horizon caps at 600 points — 25 days at hourly input, or use daily input for 30 days).
4. **Task 3 — Compare against budget:** if `projection × rate > monthly_budget × (days_remaining / days_in_month)`, fire.
5. **Task 4 — Notify:** post to a finance-and-platform escalation channel.

When workflow alerts overlap with native

Native Cost Monitors and Budget Alerts cover the *strategic* layer. Workflow alerts cover the *tactical* layer — finer cuts, faster cadence, your-team's-own-action-handoff. The two should not duplicate; if a Cost Monitor already catches "Logs ingest is way up," don't build a workflow that re-detects the same condition. Build the workflow at the cut native is silent on (per-bucket, per-team, per-cost-center).

> **Sources:** Workflows (DT docs), Predictive AI analysis (DT docs), Forecast costs with run-rate projections (DT docs) — week-over-week growth and days-until-budget-alert DQL.

8. Worked Example — End-of-Month Projection



Scenario: It's day 18 of a 30-day billing month. The platform team wants to know whether the current burn projects to exceed the monthly Log Ingest budget of 800 GiB.

Step 1 — Month-to-date consumption:

```
// Month-to-date log ingest GiB — replace the from: range with your billing-month start in the contract time zone
// @M = start of the current calendar month in the query time zone; for a contract-time-zone billing month, set explicit from:/to:
fetch dt.system.events, from:@M
| filter event.kind == "BILLING_USAGE_EVENT"
| filter event.type == "Log Management &amp;amp; Analytics - Ingest &amp;amp; Process"
| dedup event.id
| summarize { mtd_gib = sum(toDouble(billed_bytes)) / 1073741824 }
```

Suppose this returns `mtd_gib = 540`. The team has consumed 540 GiB in the first 18 days.

Step 2 — Linear projection (naive):

`projection = 540 × (30 / 18) = 900 GiB` — that's 12.5% over the 800 GiB budget. The naive projection assumes uniform daily consumption.

Step 3 — Davis-forecast projection (better):

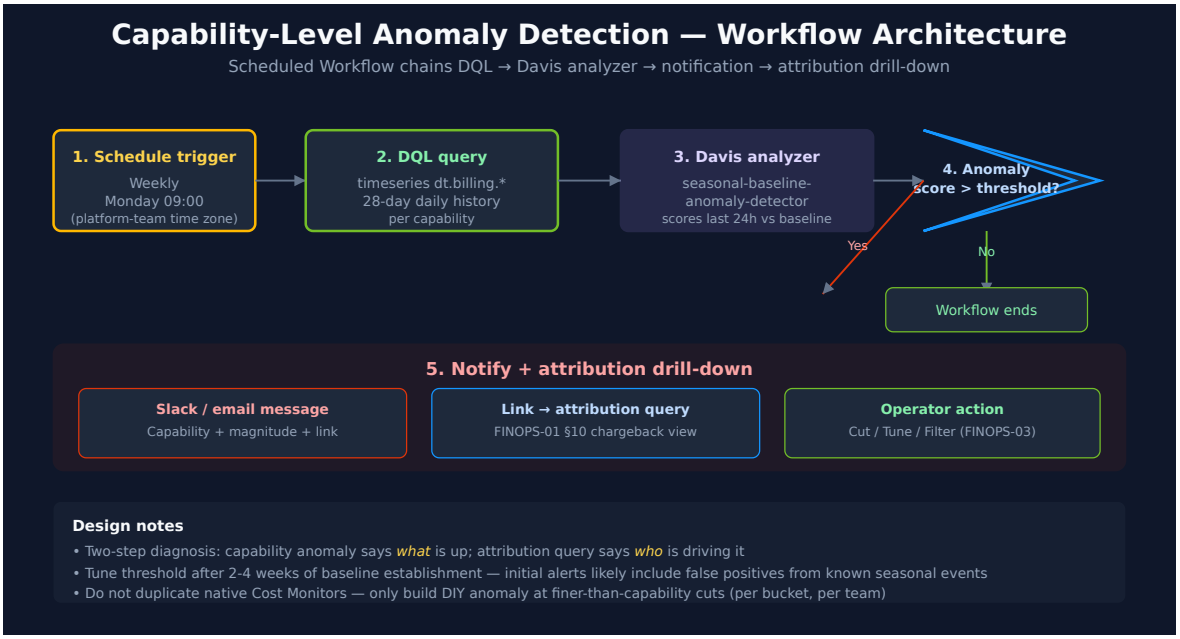
Run the 28-day input through the `timeseries-forecast` analyzer for a 12-day horizon (the rest of the month). The forecast accounts for daily / weekly seasonality (e.g., lower weekend traffic), which the naive projection ignores.

Step 4 — Decision:

In community practice, the decision rule is simple. If both projections agree the budget will be exceeded: act now. The Cut / Tune / Filter framework in FINOPS-03 covers what to do. If they disagree: investigate the forecast confidence bands — wide bands suggest the recent history is too noisy to forecast reliably (re-baseline after 1-2 more weeks; until then, manage tactically week-over-week).

> **Sources:** Predictive AI analysis (DT docs), Forecast costs with run-rate projections (DT docs) — the flat run-rate is *"your flat run-rate baseline, with no growth assumed"*.

9. Worked Example — Capability-Level Anomaly Alert



Scenario: The platform team wants its own in-tenant signal when any host-monitoring capability's daily consumption deviates from its 4-week baseline, routed through a workflow to its own channel. (Cost Monitors already check each capability daily, but deliver through Account Management — Notification Center, email, API.)

Step 1 — Build the input series (28-day daily history per capability):

```
// Daily host-hours per host capability — 28-day baseline input
timeseries {
  full_stack = sum(dt.billing.full_stack_monitoring.usage, rate:24h),
  infrastructure = sum(dt.billing.infrastructure_monitoring.usage, rate:24h),
  code = sum(dt.billing.code_monitoring.usage, rate:24h),
  k8s = sum(dt.billing.kubernetes_monitoring.usage, rate:24h)
  },
  from:-28d, interval:24h
```

Step 2 — Feed into the seasonal baseline anomaly detection analyzer, checking the latest day against the confidence band learned from the preceding days.

Step 3 — Workflow logic:

- If any capability's latest value falls outside the analyzer's confidence band (width set by `tolerance`, default 4), the workflow fires.
- The notification includes the capability name, the magnitude of deviation, and a link to the FINOPS-01 §5 per-host (or §10 per-cost-center) attribution query for drill-down.

Step 4 — Tune the threshold:

Start with the analyzer's default tolerance. After the first 2-4 alerts, judge whether each was actionable: a true cost spike, or a known event (a planned rollout, a synthetic-test enable, a one-time job)? Adjust threshold + add filter conditions (e.g., "ignore Tuesday 02:00-04:00 when the weekly cleanup job runs") to suppress known patterns.

Step 5 — Pair with bucket-level attribution:

Capability-level anomaly tells you something is up; FINOPS-01 §10 (per-cost-center attribution) tells you *which* bucket or team is driving it. Dynatrace's own cost-spike tutorial runs the same two steps: identify the capability, then attribute the spike to the entity, dashboard, workflow, or detector behind it — followed by the conversation with the owning team.

> **Sources:** Trace a cost spike to its root cause (DT docs) — *"Identify which DPS capability is driving a cost spike"*, then *"Attribute the spike to the responsible entity, dashboard, workflow, or detector."* Seasonal baseline (DT docs), Workflows (DT docs). The Step 1 input query re-executed 10/05/2026 with grouped aggregations and `24h` (29 daily points, no notifications).

10. When to Use Native vs DIY

Question	Native (Account portal)	DIY (DQL + Davis + Workflow)
Will we hit annual commit?	✓ Cost Overview forecast	(overkill)
Alert on month-to-date overshoot	✓ Budget Alerts	(overkill)
Alert on capability-level anomaly	✓ Cost Monitors	DIY only if you need per-bucket cut
Alert on per-bucket / per-team anomaly	✗ — not currently available natively	✓ DIY required
Forecast per-bucket consumption	✗ — capability-level only	✓ DIY required
Per-cost-center chargeback forecast	✗ — not currently available natively	✓ DIY required (use `_by_costcenter` series)
Day-over-day burn comparison	(Cost Overview gives weekly cut)	✓ DIY for fine-grained cut
Custom escalation routing (team → finance → exec)	(per-budget recipients; Notification Center routing by alert category)	✓ DIY workflow with conditional routing

The pattern

Adopt native first. Cost Monitors and Budget Alerts cover the strategic layer with no engineering work. **Add DIY where native is silent.** That's almost always at finer-than-capability granularity — per bucket, per team, per cost center. **Do not duplicate.** If Cost Monitors already alerts on "Logs ingest spike," don't build a workflow that re-detects the same thing — build the workflow on the per-team cut.

In community practice, the most common DIY use cases are:

1. Per-bucket weekly trend with delta-vs-baseline alert (most useful single signal)
2. Per-cost-center monthly chargeback report (delivered to finance as a scheduled notebook export)

3. Capability-level forecast with end-of-month projection (validates Cost Overview's forecast at finer granularity)

The native column moves with the product — verify current Cost Monitor and Budget capability before building a DIY pattern that may already be covered.

> **Sources:** Forecast costs with run-rate projections (DT docs) — *"Account Management already gives you a built-in forecast, but it only goes as far as environment and capability level."*; its Account-Management-vs-DQL table marks capability-level forecast, cost-center or team breakdown, and days-until-threshold as DQL-only. Cost monitors (DT docs) — *"Route different alert categories to different teams"*

11. Recommended Approach

A six-step plan for layering forecasting + anomaly detection on top of FINOPS-01's consumption visibility:

1. **Review the Budgets you already have.** Every DPS account starts with 75/90/100 thresholds notifying license administrators. Add the recipients who can act, add an earlier custom tier (e.g., 50%) if you want more lead time, and add environment- or capability-scoped budgets where one area drives cost.
2. **Point Cost Monitor notifications at the people who can act.** They are already on and check every capability daily; what you set is recipients and the forecast thresholds (week-over-week jump, total forecast — default 95%).
3. **Build one DIY weekly per-bucket trend report.** The week-over-week pattern in §5 with delta-pct sort, scheduled via Workflow, delivered to a `#cost-watch` Slack channel. This is the tactical-layer signal that most programs lack.
4. **Once the trend report has 4 weeks of history, add anomaly detection.** Run the §9 per-capability anomaly workflow against `dt.billing.*` series.
5. **Add the end-of-month projection workflow.** Combine the §8 worked example into a weekly workflow that posts the projection alongside the budget.
6. **Pair with FINOPS-01 attribution queries for drill-down.** Anomalies and projections answer *whether*; FINOPS-01 §10 answers *who*. Wire the two together so an alert link points to the attribution view.

Resist the temptation to build elaborate forecasting before basic Budget Alerts and Cost Monitors are wired. Native covers most cases; DIY adds value where native is silent, not

where it duplicates.

12. Summary

Forecasting and anomaly detection on DPS consumption operate at three layers — operational, tactical, strategic — and across two surfaces: native (Cost Monitors, Budget Alerts in Account Management) and DIY (DQL trend queries, Davis Predictive AI, Workflow burn-rate alerts). Native covers the strategic layer automatically; DIY fills in the tactical and operational layers at finer granularity than native currently exposes. The most common gap in cost programs is the tactical layer — weekly per-bucket / per-team burn-rate alerts that turn cost surprises into cost conversations early enough to act.

Next Steps

- Read **FINOPS-03** for the Cut / Tune / Filter optimization decision framework — once an alert fires, what do you actually do about it?
- Read the **WFLOW** topic series for Workflow construction patterns — scheduled triggers, DQL tasks, notification actions.
- Read the **ORGNZ** topic series for bucket and cost-center labeling — the upstream lever for meaningful per-team forecasts.
- Review the default Budgets and Cost Monitor recipients in Account Management today — both are already on, and a few minutes adding the right recipients covers most of the strategic layer.
- Open the ready-made **Usage - Overview** dashboard (Ready-made usage dashboards (DT docs)) and use the trends visible there as the starting input for your first Davis-forecast notebook.

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.